



UNCUYO
UNIVERSIDAD
NACIONAL DE CUYO

PARADIGMAS DE PROGRAMACIÓN

UNIDAD 3

Dr. Pablo Javier Vidal

Facultad de Ingeniería
Universidad Nacional de Cuyo

2026

- 1 Variables
- 2 Tipos y Ligaduras
- 3 Alcance (scope) de variables

Variables

Repaso: Valores y variables

- Las variables, los procedimientos o las constantes tienen nombres asignados por el programador
- Los valores asignados son cualquier entidad almacenable como enteros, reales o arreglos.
- Las localizaciones son lugares o zonas de memoria donde se almacenan estos valores.
- El significado de un nombre queda determinado por las propiedades o atributos asociados con el mismo

Variables y constantes

- Ejemplo: `const int n = 5`
- Esta declaración convierte a `n` en una constante entera con valor 5, es decir, **asocia** al nombre `n` el atributo de tipo de datos contante qentera y con valor 5

Variables: atributos

Variable: entidad identificada por un nombre, asociada a una dirección de memoria, un tipo de dato, un valor, un ámbito y un tiempo de vida.

Variable (nombre, dirección, tipo, valor, tiempo de vida, ámbito)

Algo más bonito y académico: Una variable es un identificador asociado a una posición de memoria capaz de almacenar un valor de un tipo determinado, con un ámbito de visibilidad y un tiempo de vida definidos durante la ejecución del programa.

Atributo: Nombres e identificadores

- Cadena de caracteres utilizado para identificar entradas en un programa
- Consideraciones de diseño:
 - ¿tienen longitud máxima?
 - ¿es sensible a mayúsculas y minúsculas?
 - ¿se permite el uso de conectores?
 - ¿hay palabras especiales (claves o reservadas)?
- Longitud: si es muy restringida afecta la legibilidad del programa. Ej:
 - Fortran: longitud máxima 6
 - Cobol: longitud máxima 30
 - Ada y Java: sin limites, todos los caracteres son significativos
 - C++: sin limites, pero los implementadores generalmente imponen uno
- Sensibilidad a mayúsculas y minúsculas: interfiere seriamente con la facilidad de lectura (readability)

Atributo: Dirección

- **Dirección:** la dirección de memoria a la que está asociada una misma variable puede estar asociada a locaciones de memoria diferentes durante la ejecución del programa, y en diferentes lugares del programa.
- **Aliasing:** dos nombres de variables diferentes pueden utilizarse para acceder a la misma locación de memoria al mismo tiempo
- El aliasing perjudica la legibilidad de los programas, ya que el programador que lee el programa debe tener presente todos los alias de una locación.
- Los alias se pueden crear a través del uso de punteros, variables por referencia.

Tipos y Ligaduras

Atributo Tipos

Variable (**nombre**, **dirección**, **tipo**, valor, tiempo de vida, ámbito)

¿Qué es un tipo?

- Un **tipo** describe un conjunto de valores y las operaciones predefinidas para esos valores.
- Al asignarle un tipo a una expresión, se delimita cuál es el conjunto de valores que esa expresión puede denotar.
- Un **sistema de tipos** provee una forma de estudiar qué valores tiene sentido usar en una operación dada.
- Por ejemplo, si x e y son valores numéricos, la expresión $x + y$ tiene sentido; si son booleanos, $x + y$ carece de sentido.
- Toda expresión en un programa puede denotar diferentes valores en distintos momentos (según el lenguaje, esto puede ocurrir en una misma ejecución o entre ejecuciones distintas).

Tipos

- La idea de tipo nos permite relacionar:
 - un conjunto de valores que pertenecen a ese tipo,
 - con las operaciones que pueden realizarse sobre esos valores.

Idea práctica

El tipo de una variable determina:

- el rango de valores que puede tomar, y
- el conjunto de operaciones definidas para esos valores.

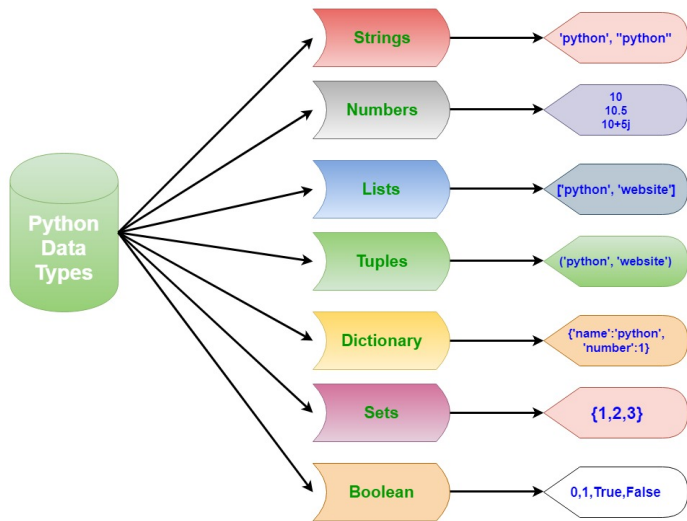
Ej.: `integer`: $[-32768., 32767]$, $\{+, -, *, /\}$

Tipos

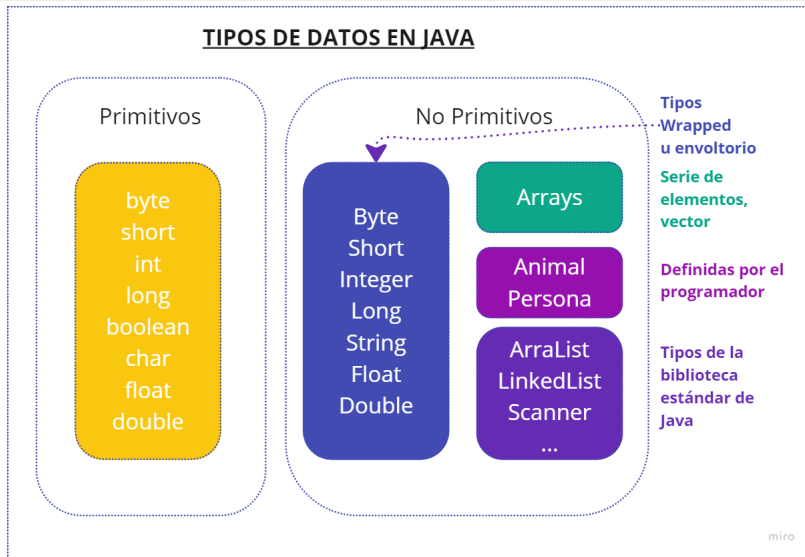
¿Para qué sirve un sistema de tipos?

- Los sistemas de tipos ayudan a construir programas correctos e impedir que se representen estados incorrectos.
- Clasifican valores y expresiones en tipos, definen cómo se manipulan y cómo interactúan entre sí.

Ejemplo



Ejemplo



Tipado estático y tipado dinámico

- **Tipado estático:** los tipos de las variables/expresiones se conocen (o se verifican) antes de ejecutar el programa, típicamente en compilación.
- Ejemplos: Java, C, C++, Rust.
- En muchos lenguajes los tipos se declaran explícitamente; en otros, se pueden inferir (por ejemplo, OCaml, Haskell, Kotlin, TypeScript en varios casos).

Tipado estático y tipado dinámico

- **Tipado dinámico:** el tipo se asocia a los valores en tiempo de ejecución, y el intérprete/runtime determina cómo operar según el valor actual.
- Una misma variable puede referenciar valores de distinto tipo en distintos momentos.
- Ejemplos: Python, JavaScript, Ruby.

Tipado fuerte o débil

- En general, se dice que un lenguaje tiene **tipado fuerte** cuando impone reglas estrictas sobre cómo se combinan los tipos.
- Suele requerir conversiones explícitas cuando se quiere operar entre tipos diferentes.
- Esto reduce ambigüedades y errores difíciles de detectar.

Tipado fuerte o débil

- Se habla de **tipado débil** cuando el lenguaje permite más conversiones implícitas o mezclas entre tipos.
- Esto puede dar más flexibilidad, pero también puede introducir comportamientos inesperados.

Tipado fuerte vs. tipado débil

- No existe una única definición universalmente aceptada de “fuerte” y “débil”.
- En general, un lenguaje se considera **débilmente tipado** si las reglas sobre qué se puede hacer con los tipos no son estrictas.
- En caso contrario, se lo considera **fuertemente tipado**.
- No debe confundirse con la distinción entre **tipado estático** y **tipado dinámico**.

Tipado nominal y estructural

- **Tipado nominal:** dos tipos son compatibles si su compatibilidad está determinada por su **nombre** (o por una relación declarada explícitamente).
- **Tipado estructural:** dos tipos son compatibles si tienen la **misma estructura** (o una estructura compatible).

■ Tipo

- E=entero
- A=vector de 1 a 10 de enteros

■ Var

- X1 = entero
- X2 = entero
- X3 = E
- V1 = vector de 1 a 10 de enteros
- V2 = A

■ Equivalencia Nominal

- X1 eqN x2
- V2 eqN v3

■ Equivalencia Estructural

- X1 eqE X2 eqE X3
- V1 eqE V2 eqE V3

Concepto de ligadura

- Una **ligadura** es la asociación entre dos cosas.
- Por ejemplo, puede darse entre un **nombre** y la cosa nombrada por él.
- También puede ser una asociación entre un **atributo** y una **entidad**.
- El **proceso de asociar un atributo a una entidad** (por ejemplo, un nombre) se conoce como **ligadura** (*binding*).
- El momento en el que se produce se denomina **tiempo de ligadura**.

¿Qué es una ligadura?

Definición

Una **ligadura** es la asociación entre un identificador (nombre) y una entidad del programa.

- Nombre $\rightarrow$ Valor
- Nombre $\rightarrow$ Dirección de memoria
- Nombre $\rightarrow$ Tipo
- Nombre $\rightarrow$ Función

Idea clave

Una variable no es solo un nombre, sino un nombre **ligado** a algo.

¿Qué es una ligadura?

Definición

Una **ligadura** es la asociación entre un identificador (nombre) y una entidad del programa.

- Nombre $\rightarrow$ Valor
- Nombre $\rightarrow$ Dirección de memoria
- Nombre $\rightarrow$ Tipo
- Nombre $\rightarrow$ Función

Idea clave

Una variable no es solo un nombre, sino un nombre **ligado** a algo.

¿Qué es una ligadura?

Definición

Una **ligadura** es la asociación entre un identificador (nombre) y una entidad del programa.

- Nombre $\rightarrow$ Valor
- Nombre $\rightarrow$ Dirección de memoria
- Nombre $\rightarrow$ Tipo
- Nombre $\rightarrow$ Función

Idea clave

Una variable no es solo un nombre, sino un nombre **ligado** a algo.

Formalización de una ligadura

Modelo formal

$$\textit{Binding} : \textit{Id} \rightarrow \textit{Entidad}$$

Entorno (Environment)

$$\textit{Env} : \textit{Id} \rightarrow \textit{Val}$$

Ejemplo

$$\textit{Env}(x) = 5$$

- Id: identificadores
- Val: valores
- Entidad: valor, tipo, dirección, función

Formalización de una ligadura

Modelo formal

$$\textit{Binding} : \textit{Id} \rightarrow \textit{Entidad}$$

Entorno (Environment)

$$\textit{Env} : \textit{Id} \rightarrow \textit{Val}$$

Ejemplo

$$\textit{Env}(x) = 5$$

- Id: identificadores
- Val: valores
- Entidad: valor, tipo, dirección, función

Formalización de una ligadura

Modelo formal

$$\textit{Binding} : \textit{Id} \rightarrow \textit{Entidad}$$

Entorno (Environment)

$$\textit{Env} : \textit{Id} \rightarrow \textit{Val}$$

Ejemplo

$$\textit{Env}(x) = 5$$

- Id: identificadores
- Val: valores
- Entidad: valor, tipo, dirección, función

Formalización de una ligadura

Modelo formal

$$\textit{Binding} : \textit{Id} \rightarrow \textit{Entidad}$$

Entorno (Environment)

$$\textit{Env} : \textit{Id} \rightarrow \textit{Val}$$

Ejemplo

$$\textit{Env}(x) = 5$$

- Id: identificadores
- Val: valores
- Entidad: valor, tipo, dirección, función

Ejemplo en código

```
x = 5  
y = x + 2
```

- $x \rightarrow 5$
- $y \rightarrow$ resultado de expresión

Interpretación

- Se crean ligaduras en el entorno
- El programa mantiene un estado interno

Ejemplo en código

```
x = 5  
y = x + 2
```

- $x \rightarrow 5$
- $y \rightarrow$ resultado de expresión

Interpretación

- Se crean ligaduras en el entorno
- El programa mantiene un estado interno

Ejemplo en código

```
x = 5  
y = x + 2
```

- $x \rightarrow 5$
- $y \rightarrow$ resultado de expresión

Interpretación

- Se crean ligaduras en el entorno
- El programa mantiene un estado interno

¿Dónde ocurre la ligadura?

- ❶ **Código fuente** (nivel sintáctico)
- ❷ **Tabla de símbolos** (compilación)
- ❸ **Memoria** (ejecución)

Idea

La ligadura no está en el texto, sino en la **interpretación del lenguaje**.

¿Dónde ocurre la ligadura?

- 1 **Código fuente** (nivel sintáctico)
- 2 **Tabla de símbolos** (compilación)
- 3 **Memoria** (ejecución)

Idea

La ligadura no está en el texto, sino en la **interpretación del lenguaje**.

Tabla de símbolos

Nombre	Tipo	Dirección	Ámbito
x	int	0x1234	local
y	int	0x1238	local

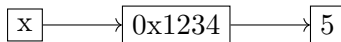
- Representa las ligaduras en compilación
- Gestiona nombres, tipos y ámbitos

Tabla de símbolos

Nombre	Tipo	Dirección	Ámbito
x	int	0x1234	local
y	int	0x1238	local

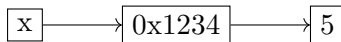
- Representa las ligaduras en compilación
- Gestiona nombres, tipos y ámbitos

Ligadura en memoria



- Nombre $\rightarrow$ Dirección
- Dirección $\rightarrow$ Valor

Ligadura en memoria



- Nombre → Dirección
- Dirección → Valor

Ligadura y ámbito

```
x = 5
```

```
def f():  
    x = 10
```

- Dos ligaduras distintas para x
- Dependen del ámbito (scope)

Ligadura y ámbito

```
x = 5
```

```
def f():  
    x = 10
```

- Dos ligaduras distintas para x
- Dependen del ámbito (scope)

Resumen

- Ligadura = nombre $\rightarrow$ entidad
- Se formaliza como función
- Se implementa en:
 - Tabla de símbolos
 - Memoria
- Se evidencia en:
 - Declaraciones
 - Asignaciones
 - Ámbitos

Concepto clave

Entender ligaduras permite comprender:

- variables
- scope
- tiempo de vida
- memoria

Resumen

- Ligadura = nombre $\rightarrow$ entidad
- Se formaliza como función
- Se implementa en:
 - Tabla de símbolos
 - Memoria
- Se evidencia en:
 - Declaraciones
 - Asignaciones
 - Ámbitos

Concepto clave

Entender ligaduras permite comprender:

- variables
- scope
- tiempo de vida
- memoria

Tiempo de ligadura

- El **tiempo de ligadura** es el momento en el cual se crea una ligadura, o bien cuando se toma una decisión de implementación.
- Existen distintos momentos en los que estas decisiones pueden quedar vinculadas.
- Dependiendo del momento de asociación, la ligadura puede ser:
 - **Estática** (tiempo de compilación)
 - **Dinámica** (tiempo de ejecución)

Tiempo de ligadura

- Solo en **ejecución** se produce lo que se denomina **ligadura dinámica**.
- En los demás momentos, se habla de **ligadura estática**.

Ligadura del tipo a una variable: dinámica

- La correspondencia se establece al asignar valores o al invocar métodos.
- Soporta la evolución de los programas sin necesidad de recompilar.
- Tiene un costo en tiempo de ejecución.
- Puede especificarse a través de la asignación:
 - `ls = [2,4,6]` → `ls` es una lista
 - `ls = 17.3` → `ls` es un valor real

Ventajas y desventajas

- **Ventajas:** flexibilidad (unidades de programa genéricas).
- **Desventajas:** mayor costo (chequeo dinámico de tipos e interpretación) y detección de errores más compleja, en tiempo de ejecución.

Ligadura estática vs. ligadura dinámica

Ligadura estática

La asociación queda resuelta **antes de ejecutar** el programa, típicamente en compilación.

- tipos de variables
- alcance léxico
- sobrecarga resuelta por el compilador

Ligadura dinámica

La asociación se decide **durante la ejecución**.

- nombres ligados a objetos en runtime
- despacho dinámico de métodos
- variables lógicas unificadas durante la consulta

Regla rápida

Si la relación se fija **antes de correr**, es estática. Si se resuelve **mientras corre**, es dinámica.

Ejemplos típicos de tiempos de ligadura

Qué se liga	Momento usual	Ejemplo
Nombre → tipo	Estático	<code>int x;</code> en C/Java
Nombre → ámbito	Estático	alcance léxico en Haskell, C, JavaScript
Nombre → valor/objeto	Dinámico	<code>x = 5 / x = "hola"</code> en Python
Llamada → método	Dinámico	métodos virtuales en Java
Variable lógica → término	Dinámico	unificación en Prolog

Haskell: predominio de ligadura estática

```
doble :: Int -> Int  
doble x = x * 2
```

```
main =  
  let y = 10  
  in doble y
```

- **doble** queda ligado a una función.
- **x** e **y** tienen **ámbito léxico**.
- El tipo `Int -> Int` se conoce **antes de ejecutar**.
- La ligadura nombre→tipo es **estática**.

Idea

En Haskell, el valor concreto se usa en runtime, pero gran parte de las ligaduras relevantes (tipo, alcance, definición) se fijan estáticamente.

Prolog: ligadura dinámica por unificación

```
padre(juan , maria ).  
padre(maria , pedro ).
```

```
abuelo(X, Y) :-  
    padre(X, Z) ,  
    padre(Z, Y).
```

Consulta:

```
?- abuelo(A, pedro ).  
A = juan .
```

- X, Y, Z, A son **variables lógicas**.
- No reciben un valor por asignación tradicional.
- Se ligan **durante la resolución** de la consulta.
- La unificación produce una ligadura **dinámica**.

Lectura

La pregunta no dice *cómo* calcular, sino *qué relación* debe cumplirse; el intérprete liga variables al buscar una solución.

Java y C: ejemplos clásicos

Java

```
class A {
    void saludar() {
        System.out.println("A");
    }
}
class B extends A {
    void saludar() {
        System.out.println("B");
    }
}
A x = new B();
x.saludar();
```

- x tiene tipo estático A.
- El método ejecutado se decide en runtime.
- x.saludar() usa **ligadura dinámica**.

C

```
int doble(int x) {
    return x * 2;
}

int main() {
    int y = 10;
    return doble(y);
}
```

- tipos y nombres resueltos por el compilador
- llamada normal a función: **estática**
- con punteros a función puede haber decisión en runtime

JavaScript y Python: ejemplos más dinámicos

JavaScript

```
let x = 5;  
x = "hola";
```

```
const obj = {  
  f() { return "JS"; }  
};  
obj.f();
```

- **let** tiene alcance léxico: rasgo estático.
- El nombre **x** puede ligarse a valores de distinta clase en runtime.
- Predominan ligaduras **dinámicas de valor**.

Python

```
x = 5  
x = "hola"
```

```
def f(a):  
    return a + a
```

- Los nombres se ligan a objetos en ejecución.
- No se fija un tipo de variable en compilación.
- **x** cambia de objeto ligado: **dinámico**.

Comparación rápida por lenguaje

Lenguaje	Predominio	Ejemplo de ligadura
Haskell	Estática	nombre→tipo, alcance léxico, definiciones inmutables
Prolog	Dinámica	variable lógica→término por unificación
Java	Mixta	tipos estáticos + despacho dinámico de métodos
C	Mayormente estática	tipos, funciones y direcciones resueltos por compilación/enlace
JavaScript	Mixta, más dinámica	alcance léxico con <code>let</code> , valores resueltos en runtime
Python	Dinámica	nombres ligados a objetos durante la ejecución

Resolución de ligaduras

	Comprobación Estática	Comprobación Dinámica
Ligadura Estática	Garantía de corrección Interpretación Inflexible	Combinación no valida (sin sentido)
Ligadura Dinámica	Garantía de corrección Interpretación Flexible	No hay garantía de corrección Interpretación Flexible

Idea general: ligadura vs comprobación

Ligadura (binding):

- Asociación entre un nombre y una entidad.
- Ejemplos:
 - nombre $\rightarrow$ tipo
 - nombre $\rightarrow$ valor
 - nombre $\rightarrow$ dirección de memoria
 - llamada $\rightarrow$ implementación de método

Comprobación (checking):

- Verificación de que una operación sea válida.
- Ejemplos:
 - verificar si un cast es correcto
 - verificar si un índice está dentro de rango
 - verificar si un objeto no es `null`

Idea clave: el momento de la ligadura y el momento de la comprobación **no tienen por qué coincidir**.

Casos posibles

- **Ligadura estática:** la asociación se decide antes de ejecutar el programa.
- **Ligadura dinámica:** la asociación se decide durante la ejecución.
- **Comprobación estática:** la validez se verifica antes de ejecutar.
- **Comprobación dinámica:** la validez se verifica en tiempo de ejecución.

Consecuencia importante:

- Puede haber **ligadura estática con comprobación dinámica**.
- Puede haber **ligadura dinámica con comprobación estática** en ciertos contextos restringidos.
- No conviene pensar “estático” y “dinámico” como bloques indivisibles.

Ejemplo 1: ligadura estática + comprobación dinámica (cast en Java)

```
Object x = "hola";  
String s = (String) x;  
System.out.println(s.length());
```

Qué ocurre aquí:

- El nombre `x` tiene tipo estático `Object`.
- La expresión `(String) x` es aceptada por el compilador.
- Pero la validez concreta del cast se comprueba en ejecución.

Si cambiamos el valor:

```
Object x = 10;  
String s = (String) x;    // ClassCastException
```

Conclusión: la ligadura puede ser estática y la comprobación, dinámica.

Ejemplo 2: ligadura estática + comprobación dinámica (índices)

```
int[] v = new int[5];  
v[2] = 10;  
v[10] = 20;    // ArrayIndexOutOfBoundsException
```

Observación:

- El compilador conoce que *v* es un arreglo de enteros.
- Sin embargo, no siempre puede garantizar que el índice usado será válido.
- El chequeo del rango se realiza en runtime.

Idea:

- Ligadura del nombre y del tipo: estática.
- Comprobación del acceso: dinámica.

Ejemplo 3: ligadura estática + comprobación dinámica (null)

```
String s = null;  
System.out.println(s.length());    // NullPointerException
```

Análisis:

- El compilador sabe que **s** es de tipo **String**.
- Pero el hecho de que la referencia apunte a un objeto válido se confirma recién al ejecutar.
- Nuevamente, la comprobación ocurre dinámicamente.

Ejemplo 4: ligadura dinámica + comprobación dinámica (Python)

```
x = "hola"
print(x.upper())

x = 10
print(x.upper())    # AttributeError
```

Qué muestra este ejemplo:

- El nombre `x` puede quedar asociado a objetos de distinto tipo durante la ejecución.
- La operación válida sobre `x` se decide observando el valor actual.
- Tanto la ligadura útil para la operación como la comprobación ocurren en runtime.

Ejemplo 5: polimorfismo y binding dinámico

```
class Animal {  
    void hablar()  
    { System.out.println("sonido"); }  
}  
  
class Perro extends Animal {  
    @Override  
    void hablar()  
    { System.out.println("guau"); }  
}  
  
Animal a = new Perro();  
a.hablar();
```

Análisis:

- El tipo estático de `a` es `Animal`. El objeto real en ejecución es `Perro`.
- La implementación concreta del método se decide en runtime.

Esto es: **ligadura dinámica de métodos** o **dynamic dispatch**.

Polimorfismo: relación con las ligaduras

Polimorfismo por sobrescritura (override):

- Suele implicar **ligadura dinámica**.
- La llamada al método se resuelve en ejecución.

Polimorfismo por sobrecarga (overload):

- Suele implicar **ligadura estática**.
- El compilador decide qué versión invocar según los tipos visibles.

Entonces:

- `override` → típicamente binding dinámico
- `overload` → típicamente binding estático

Ejemplo breve: overload vs override

Sobrecarga: decisión estática

```
class Ejemplo {  
    void f(int x)      { System.out.println("int"); }  
    void f(double x) { System.out.println("double"); }  
}
```

```
Ejemplo e = new Ejemplo();  
e.f(3);      // el compilador elige f(int)
```

Sobrescritura: decisión dinámica

```
Animal a = new Perro();  
a.hablar(); // en runtime se ejecuta hablar() de Perro
```

Conexión entre estos conceptos

Concepto	Pregunta que responde
Ligadura	¿Cuándo se asocia un nombre con una entidad?
Comprobación	¿Cuándo se verifica si una operación es válida?
Scope	¿Dónde se busca un nombre?
Polimorfismo	¿Puede una misma referencia usarse con distintas implementaciones?
Type inference	¿El tipo se escribe o se deduce?
Strong/Weak typing	¿Qué tan estrictas son las restricciones del sistema de tipos?

Tabla corregida: ligadura vs comprobación

	Comprobación Estática	Comprobación Dinámica
Ligadura Estática	Garantía fuerte de corrección Menor flexibilidad	Combinación válida La asociación se fija antes, pero se verifica en runtime Ej.: casts, null checks, rangos
Ligadura Dinámica	Posible en contextos restringidos Flexibilidad con ciertos controles	Alta flexibilidad Menores garantías previas Verificación en ejecución

Conclusión

- La celda “Ligadura Estática + Comprobación Dinámica” **sí tiene sentido**.
- Muchos lenguajes reales combinan decisiones estáticas con verificaciones dinámicas.
- El polimorfismo por sobrescritura se asocia normalmente con **binding dinámico**.
- La inferencia de tipos no implica tipado dinámico.
- Strong/weak typing y static/dynamic typing describen dimensiones distintas del lenguaje.

Idea final: para entender un lenguaje conviene separar:

- cuándo se liga
- cuándo se comprueba
- dónde se busca
- qué tan estricto es el sistema de tipos

Ideas para cerrar

- 1 Ligadura = asociación entre nombre y entidad.
- 2 **Estática**: se fija antes de ejecutar.
- 3 **Dinámica**: se resuelve durante la ejecución.
- 4 La mayoría de los lenguajes reales combinan ambas.

Lectura final por paradigma

- funcional (Haskell): más estático
- lógico (Prolog): ligadura dinámica por unificación
- orientado a objetos (Java): mezcla de tipo estático y despacho dinámico
- scripting (Python/JavaScript): mayor dinamismo en valores

Alcance (scope) de variables

Entidad / Variable: atributos semánticos

- **Nombre:** identificador en el código.
- **Dirección / ubicación:** stack, heap, registro, segmento de datos, etc.
- **Tipo:** valores permitidos + operaciones válidas.
- **Valor:** contenido actual.
- **Tiempo de vida (lifetime):** intervalo en ejecución donde la entidad existe.
- **Ámbito/alcance:** dónde el nombre es visible y se vincula a esa entidad.

Idea clave

Ámbito se observa en el texto del programa; **tiempo de vida** depende de la ejecución.

Ámbito vs Alcance (idea práctica)

Ámbito (scope)

Región del programa donde un identificador es **visible** y puede ser referenciado.

Alcance (reach / alcance efectivo)

Conjunto de puntos del programa donde una declaración **aplica** (donde el nombre “hace referencia” a esa entidad).

En la práctica (para la materia)

A menudo se usan como sinónimos: “**zona donde se puede usar el nombre**”.

Alcance de una variable

- Una variable tiene una **ligadura activa** para una sentencia si puede ser referenciada en ella.
- El alcance de una ligadura puede determinarse:
 - Estáticamente
 - Dinámicamente

Regla de alcance

Determina cómo las referencias a variables declaradas fuera del subprograma o bloque en ejecución se asocian con sus declaraciones y, por lo tanto, con sus atributos.

Alcance estático (léxico)

- En un lenguaje con **alcance estático** o **léxico**, la ligadura se determina en tiempo de compilación a partir de la examinación del texto del programa.
- Habitualmente, la ligadura se resuelve buscando la declaración cuyo bloque esté más cerca de ese punto del programa.

Alcance dinámico

- En el **alcance dinámico**, la ligadura se resuelve siguiendo la secuencia de llamadas, no la estructura textual del programa.
- Por eso, solo puede determinarse en tiempo de ejecución.

Alcance léxico vs alcance dinámico

Alcance léxico (estático)

Se determina por la **estructura del código** (bloques anidados). Ej.: C, Java, Haskell.

Alcance dinámico

Se determina por la **pila de llamadas** en tiempo de ejecución (quién llamó a quién).

Consecuencia

En alcance dinámico, el mismo nombre puede referir a **variables distintas** según el camino de ejecución.

Shadowing (sombras) y binding

- **Shadowing**: una declaración interna con el mismo nombre **oculta** la externa.
- **Binding**: decisión de qué entidad corresponde a un nombre en un punto del programa.
- **Binding estático**: se resuelve por el compilador (típico en scope léxico).
- **Binding dinámico**: se resuelve en ejecución (típico en scope dinámico).

Regla mental

Para scope léxico: “la declaración visible más cercana en el texto del programa”.